Bangladesh Telecom Market Project

A plain-English walkthrough of every cell, library and model in **Telecom_BD_Project.ipynb**.

This guide is written for someone who is comfortable with basic Python but has not yet built an end-to-end machine-learning project. Every library is introduced before it appears, every model is described twice — once as a one-line definition and once as an everyday-language intuition — and every cell in the notebook is summarised in order. A short note on deep learning and on the privacy concept of *machine unlearning* is included at the end.

Open the notebook side by side with this document and read each section just before you run the matching cell.

Part A — The big picture

A.1 What the project does

The notebook takes a messy financial spreadsheet describing the Bangladeshi mobile-phone market between 2008 and 2015 and walks it through a full data-science workflow. Concretely we:

- **Reshape** the raw wide-format CSV (months as columns) into a tidy table where every row is one operator-month observation.
- **Explore** the data visually and statistically (EDA).
- **Engineer features** that a linear model can learn from (a calendar time index, a previous-month lag, etc.).
- **Train regression models** (Linear, Ridge, Lasso) to predict the monthly subscriber count of an operator.
- **Cluster** operator-month behaviour with KMeans and visualise it with PCA.
- **Demonstrate approximate machine unlearning** by removing one operator's influence from the trained Lasso via a single Newton / influence-function update — no retraining — simulating a real data-deletion request.

A.2 The dataset

File: *GP Model.xlsx — Telecom Mkt. in Bangladesh.csv*. The sheet was produced as a market-modelling workbook for Grameenphone, so the raw shape is wide: each row is an operator (Grameen, Banglalink, Robi, Airtel, Citycell, Teletalk, Sector total) and each column is one calendar month. After reshaping we get a tidy table with one row per (operator, month) and the following fields:

Column	Meaning
date	First-of-the-month timestamp
year, month	Calendar year and month number (1–12)
operator	Telecom operator name (categorical)
subscribers_mn	Number of subscribers in millions (the regression target)
sector_total	Total subscribers across all operators that month
market_share	subscribers_mn / sector_total
net_addition	Month-on-month change in subscribers
growth_rate	Percentage change in subscribers vs previous month
time_index	Months since January 2008 (a simple linear-time feature)
lag_1	The previous month's subscriber count for the same operator

A.3 Workflow at a glance

Read CSV → reshape → clean → EDA → feature engineering → train regression models → evaluate → KMeans + PCA → approximate machine-unlearning demo. Each step has its own notebook section (numbered 1–10 inside the notebook itself).

Part B — Libraries explained

Every line of code in the notebook comes from one of the libraries below. You can think of each library as a specialised toolbox.

NumPy (numpy, abbreviated np)

What it is. The foundational numerical library for Python. Provides the *ndarray* — a fixed-size, fast array of numbers — and a huge collection of mathematical functions that operate on those arrays.

Intuition. Plain Python lists are flexible but slow. NumPy is what makes vector/matrix operations in Python as fast as Fortran or C. Whenever a library says “give me a matrix of features”, behind the scenes it is a NumPy array.

pandas (pd)

What it is. A data-manipulation library built on top of NumPy. Its main object is the *DataFrame* — basically a spreadsheet with named columns and a row index — and it gives you familiar operations such as filtering, joining, grouping, and time-series resampling.

Intuition. If Excel and SQL had a child that lives inside Python, that child would be pandas. Whenever we read a CSV, melt, group or merge in this notebook, that is pandas at work.

Matplotlib (matplotlib.pyplot as plt)

What it is. The original Python plotting library. Low-level but very flexible: you build a figure, add axes, then add lines, bars, scatters, etc.

Intuition. Matplotlib is the canvas. Almost every other Python plot library — including seaborn — is a thin wrapper that ultimately draws onto a matplotlib figure.

seaborn (sns)

What it is. A statistical-plotting library that sits on top of matplotlib. It produces good-looking plots from a single function call and integrates directly with pandas DataFrames.

Intuition. seaborn is to matplotlib roughly what Excel charts are to drawing rectangles by hand: opinionated defaults, sensible colours, and shortcut functions like *sns.boxplot*, *sns.heatmap*, *sns.scatterplot*.

scikit-learn (sklearn)

What it is. The standard machine-learning library for Python. Provides a unified API — every model has *.fit(X, y)* and *.predict(X)* — across dozens of algorithms.

Intuition. If pandas is your spreadsheet, scikit-learn is your modelling lab. The sub-modules used in the notebook are listed below.

B.1 The scikit-learn sub-modules we use

- **model_selection** — train/test split, cross-validation.
- **preprocessing** — StandardScaler (rescales numbers to mean 0, variance 1), OneHotEncoder (converts categories to columns of 0/1).
- **compose.ColumnTransformer** — apply different transforms to different columns in one shot.
- **pipeline.Pipeline** — chain a transformer and a model so that *.fit()* trains both and *.predict()* applies both.
- **linear_model** — Linear, Ridge, Lasso regression.

- **metrics** — r2_score, mean_absolute_error, mean_squared_error.
- **cluster.KMeans** — the unsupervised clustering algorithm.
- **decomposition.PCA** — principal component analysis for dimensionality reduction.
- **base.clone / BaseEstimator** — utilities for building and copying estimators (used when wrapping the unlearned weights into a Pipeline-compatible object).

Part C — Concepts you need

Supervised vs unsupervised learning

Supervised learning has a target column (labels). You show the model many (X, y) pairs and ask it to learn the mapping X → y. Regression (predict a number) and classification (predict a category) are both supervised.

Unsupervised learning has no target column. The model finds structure in X alone — clusters of similar rows, low-dimensional axes that summarise the data, anomalies. KMeans and PCA in §8 are unsupervised.

Regression vs classification

Regression: the target y is a real number (subscriber count, house price, temperature).

Classification: the target y is a category (spam/not-spam, churn/no-churn). This notebook does regression only.

Train/test split

We hide a fraction of the data (here 25%) from the model during training and use it to estimate how well the model generalises to new months.

Intuition: it is the difference between solving the exam questions you have already seen and solving questions you have not seen. The second is what matters.

Cross-validation (CV)

Instead of a single hold-out set, the data is split into k folds (k=5 in the notebook). The model is trained on k-1 folds and tested on the remaining one; this is repeated k times and the scores averaged.

Intuition: it is like grading a student on five different exams instead of one — a much more reliable measure of skill.

Feature scaling / standardisation

StandardScaler subtracts the mean and divides by the standard deviation of every numeric feature. Result: each feature has mean 0 and variance 1.

Intuition: *time_index* ranges from 0 to ~95, while *market_share* is between 0 and 1. Without scaling, a linear model with regularisation would unfairly punish the smaller-range feature. After scaling all features speak the same language.

One-hot encoding

A categorical column (here *operator*) is expanded into one 0/1 column per category.

Intuition: a regression model cannot multiply by the string “Robi”. One-hot encoding turns “Robi” into the vector (0,0,1,0,0,0) which the model can use.

Lag feature

lag_1 is the value of the target one time-step ago. We include it as a feature because subscriber counts are extremely auto-correlated: next month’s count is mostly this month’s count plus a small change.

Intuition: if you want to forecast tomorrow’s temperature, today’s temperature is by far the most useful input.

Regularisation: L1 (Lasso) vs L2 (Ridge)

Both add a penalty to the loss function so that the model is discouraged from giving any feature an unreasonably large coefficient. L2 penalises the sum of squared coefficients (shrinks them smoothly). L1 penalises the sum of absolute coefficients (can push some all the way to zero).

Intuition: Ridge keeps everyone in the band but lowers the volume. Lasso silences the quiet instruments completely. Lasso therefore doubles as a feature-selection method.

Evaluation metrics

R^2 (coefficient of determination): 1.0 = perfect, 0.0 = no better than predicting the mean of y, negative = worse than that. Interpretable as “fraction of variance explained”.

MAE (mean absolute error): the average absolute difference between prediction and actual. Same unit as y, easy to read aloud. **RMSE** is the square root of the mean squared error — same unit as y but punishes large errors more heavily than MAE.

Residual

Actual – predicted. Plotting residuals against fitted values is the standard linear-regression diagnostic: a structured pattern (curve, funnel) means the model is missing something.

Outlier

A data point that lies far away from the bulk of the distribution. Boxplots flag them automatically (the lone dots beyond the whiskers). Outliers can be genuine extreme events (Airtel’s mid-period drop) or data-entry errors.

Inertia (for KMeans)

The sum of squared distances from every point to its assigned cluster centre. Smaller is tighter. We watch how inertia falls as k increases and pick the “elbow” — the k after which extra clusters barely help.

Part D — Models used in this notebook

D.1 Linear Regression

Definition. Linear regression assumes the target y is a weighted sum of the features plus an intercept: y ≈ b■ + b■·x■ + b■·x■ + … + b■·x■. It picks the coefficients (the b's) that minimise the sum of squared errors on the training data.

Intuition. Imagine drawing the straight line (or flat hyperplane in higher dimensions) that gets as close as possible to all training points at once. Fast, interpretable, and a strong baseline.

D.2 Ridge Regression

Definition. Linear regression with an L2 penalty: minimise (sum of squared errors) + alpha · (sum of squared coefficients). The hyper-parameter *alpha* controls how strongly we shrink.

Intuition. Vanilla linear regression is happy to assign a giant coefficient to any feature that happens to fit the training noise. Ridge pulls all coefficients toward zero, which usually generalises better.

D.3 Lasso Regression

Definition. Linear regression with an L1 penalty: (sum of squared errors) + alpha · (sum of absolute coefficients). The L1 penalty has a geometric property that drives weak coefficients exactly to zero.

Intuition. Lasso is like Ridge plus automatic feature selection: weak features are eliminated entirely from the model. The trade-off is that Lasso can be unstable when several features are highly correlated.

D.4 KMeans clustering

Definition. Pick k (the number of clusters). Initialise k centres randomly. Repeat: (1) assign every point to its nearest centre, (2) move every centre to the mean of its assigned points. Stop when assignments no longer change. The objective minimised is the inertia — the sum of squared distances from each point to its centre.

Intuition. It is k iterative gravitational pulls. Points snap to whichever centre they are closest to; centres then drift to the centre-of-mass of their followers. Repeat until stable.

Limitations. KMeans likes spherical, equally-sized clusters and needs you to pick k in advance. The elbow plot is the standard heuristic for choosing k. For non-spherical or density-based clusters, algorithms such as DBSCAN or hierarchical clustering are better.

D.5 Principal Component Analysis (PCA)

Definition. PCA finds the directions (linear combinations of the original features) along which the data has the highest variance, ranks them, and lets you project the data onto the first few. With *n_components=2* we keep only the two strongest directions.

Intuition. Imagine a cloud of points in 4-D feature space. PCA spins the axes so the first axis points along the long axis of the cloud, the second along the next-longest perpendicular direction, and so on. Discarding the short axes lets you draw the cloud in 2-D with minimal information loss — perfect for a scatter plot.

D.6 A note on deep learning

What deep learning is. A family of models built from many layers of artificial "neurons" — small functions that linearly combine their inputs and apply a non-linear activation. Stacking enough layers

lets the model approximate extremely complex relationships, which is what powers modern image and language models.

Why we did not use it here. The dataset has roughly 570 rows after reshaping and only a handful of features. Deep networks shine when you have hundreds of thousands of rows or unstructured inputs (text, images, audio). On a small structured table they tend to underperform a properly regularised linear model, while being slower, harder to interpret, and easier to overfit. The course syllabus also reserves deep learning for follow-up courses.

What a deep-learning version would look like. A feed-forward network with one or two hidden layers, the same X matrix on input and the same subscriber count on output, trained with the Adam optimiser and an MSE loss. You would build it with PyTorch or TensorFlow rather than scikit-learn.

Part E — Machine unlearning, in plain words

E.1 The problem

Regulations such as the EU's GDPR and similar privacy laws give individuals (or organisations) the right to demand that their data be deleted. Deleting the rows from the database is easy. The harder question is: **what about the AI model that was already trained on those rows?** The model's weights still encode information learnt from that data. *Machine unlearning* is the field that studies how to remove that residual influence.

E.2 Two families of methods

- **Exact unlearning.** Retrain the model from scratch on the surviving data. Provably leaves zero trace of the forgotten rows but is computationally expensive at production scale. Used as the reference baseline against which approximate methods are measured.
- **Approximate unlearning.** Cheaper algorithms that estimate and subtract the influence of the forget set without a full retrain. The three best-known approaches are **influence functions / Newton-step updates** (use second-order calculus to estimate each row's contribution to the loss and remove it in one closed-form step), **SISA** (shard the data, train independent sub-models, retrain only the affected shard), and **gradient-based unlearning** (take undo-gradient steps iteratively). This notebook implements the first option.

E.3 The scenario in §9 of the notebook

We pick an industry — a **telecom-analytics consultancy** — that has trained a Lasso regression model on the full dataset and sells subscriber forecasts to all operators. One operator (Airtel) invokes its data-deletion rights. We must update the model so that:

- Its predictions on the **retained** operators stay accurate (utility preserved).
- Its parameters no longer carry an Airtel-specific contribution (forgetting confirmed).

Instead of retraining from scratch, we apply a single **Newton / influence-function** update:

```
theta_new = theta_old + inv(H) * sum_{i in F} grad_i(theta_old)
```

where *F* is the forget set (all Airtel rows), *grad_i* is the per-row squared-error gradient and *H* is the (regularised) Hessian of the training loss. Intuitively, we estimate how much each forgotten row pulled the parameters during training and take one Newton step in the opposite direction to undo that pull. We then compare metrics, coefficients, and predicted trajectories before and after.

Honest caveat. Because the Lasso's strongest feature is *lag_1* — each operator's own previous-month subscriber count — most of the Airtel-specific signal at inference time lives in the input, not in the weights. The Newton step removes whatever Airtel information the parameters had absorbed (mainly the operator one-hot terms), while the lag-driven prediction component is unavoidable as long as Airtel's lagged value is fed to the model at inference. That is a property of the dataset, not of the method.

Part F — Cell-by-cell walkthrough

Cells are numbered 0–43 in the order they appear in the notebook. Markdown cells are headings; code cells contain Python. Each entry below summarises what the cell does and why.

Cell 0 — markdown: Project title and ten-point outline

Sets the stage: project name, course, and the list of steps the notebook will perform.

Cell 1 — markdown: Heading — §1 Imports & configuration

Section divider only.

Cell 2 — code: Import every library used and set global options

Pulls in numpy/pandas (data), matplotlib/seaborn (plots), and the scikit-learn pieces needed for splitting, preprocessing, modelling, metrics, clustering and PCA. Sets a seaborn theme, a default figure size, allows pandas to print up to 30 columns, and fixes the NumPy random seed so the results are reproducible.

Cell 3 — markdown: Heading — §2 Load the raw CSV

Explains that the source spreadsheet is in wide format and that we will read it without a header so we can pick out only the rows that contain subscriber counts.

Cell 4 — code: Read the CSV into a raw DataFrame

pd.read_csv(..., header=None) loads the whole file as untyped strings. We print the raw shape and the first 12 rows so we can confirm we are looking at the right block of cells.

Cell 5 — markdown: Heading — §3 Reshape into a tidy long format

States which rows of the CSV contain the data we care about (rows 4–10 are the seven operators, columns 2–97 are the 96 months of Jan-2008 to Dec-2015).

Cell 6 — code: Slice the data block and label its rows and columns

data_block = raw.iloc[4:11, 2:98] picks out the 7×96 rectangle of subscriber numbers. We then attach the operator names as the row index, build a proper monthly DatetimeIndex for the columns, and use *pd.to_numeric(errors='coerce')* to turn any stray strings into NaN.

Cell 7 — code: Melt to long (tidy) format and add time features

.melt() turns a wide DataFrame into a long one: one row per (operator, month) instead of one row per operator. We then derive *year*, *month*, and a *time_index* that counts months since January 2008.

Cell 8 — markdown: Heading — §4 Data preprocessing

Explains that the *Sector* row is a sum of the others and must be separated out before modelling, otherwise it would leak the answer into the features.

Cell 9 — code: Split off the Sector row, attach it back as a feature, check missingness

We pull out the Sector row, rename its subscriber column to *sector_total*, and merge it back onto the operator rows so every (operator, month) record knows the sector size that month. Printing *isna().sum()* tells us how many missing cells each column still has.

Cell 10 — code: Drop missing rows, derive market_share, net_addition, growth_rate

Rows missing the target are useless, so we drop them. Then we compute market share, month-on-month net additions (*groupby('operator').diff()*) and growth rates (*.pct_change()*).

Cell 11 — markdown: Heading — §5 Exploratory Data Analysis (EDA)

EDA is the look-before-you-leap step: understand the data before fitting any model.

Cell 12 — code: Summary statistics (.describe)

op_df.describe(include='all').T shows count, mean, standard deviation, min, max and quartiles for every column.

Cell 13 — markdown: Heading — §5.2 Subscriber growth over time

Cell 14 — code: Line plot: subscribers per operator vs date

A separate line per operator on the same axes makes the market structure obvious — Grameenphone runs away, the others bunch together.

Cell 15 — markdown: Heading — §5.3 Market-share evolution

Cell 16 — code: Stacked area chart of market share through time

pivot_table reshapes the data into one column per operator, then *.plot.area()* stacks the shares so they always sum to 1.

Cell 17 — markdown: Heading — §5.4 Distributions of the numeric features

Cell 18 — code: Histograms of each numeric feature

Tells us at a glance whether a feature is roughly Gaussian (subscriber count), heavily skewed (growth rate) or close to uniform.

Cell 19 — markdown: Heading — §5.5 Correlation heatmap

Cell 20 — code: Pearson correlation matrix as a heatmap

op_df[...].corr() computes the linear correlation between every pair of numeric columns. *sns.heatmap* draws it with red-blue colouring centred on 0. Strong red = positive correlation, strong blue = negative.

Cell 21 — markdown: Heading — §5.6 Outlier check — net additions by operator

Cell 22 — code: Boxplot of monthly net additions by operator

Each box shows the middle 50% of monthly net additions; whiskers show the typical range and individual dots are outliers. A useful sanity check before regression.

Cell 23 — markdown: Heading — §6 Feature engineering for regression

Defines the target (*subscribers_mn*) and the candidate features.

Cell 24 — code: Create lag_1 and build X, y

Sort by operator and date so that *shift(1)* picks each operator's previous month. Drop the very first month of each operator (no lag available), then collect the chosen columns into X and the target into y.

Cell 25 — markdown: Heading — §7 Regression modelling

Introduces the three linear models we will compare and the metrics we will use.

Cell 26 — code: Build the ColumnTransformer, train/test split, train three pipelines, gather metrics

A *ColumnTransformer* sends the numeric columns through StandardScaler and the operator column through OneHotEncoder. We wrap each model (Linear, Ridge, Lasso) inside a Pipeline together with a fresh preprocessor. *train_test_split* randomly hides 25% of the rows. We fit, predict, score (R^2, MAE, RMSE), and also run 5-fold cross-validation to check stability. Results land in a tidy DataFrame.

Cell 27 — markdown: Heading — §7.1 Residual diagnostics for the best model

Cell 28 — code: Residuals-vs-fitted and predicted-vs-actual scatter plots

If the residuals are a featureless cloud around 0, the linear assumption is fine. The predicted-vs-actual plot should hug the 45° line.

Cell 29 — markdown: Heading — §7.2 Coefficient comparison: Ridge vs Lasso

Cell 30 — code: Extract coefficients from each fitted Pipeline

coef_table() walks into the pipeline, finds the trained preprocessor, asks the OneHotEncoder for the column names it produced, and pairs them with the model coefficients. Useful for seeing how Lasso zeroes out weak features.

Cell 31 — markdown: Heading — §8 Unsupervised learning

Switches from supervised modelling to clustering and dimensionality reduction.

Cell 32 — code: Standardise the features and run the elbow plot

We standardise four behavioural features and run KMeans for k = 1..8. The inertia (within-cluster sum of squares) is plotted against k. The “elbow” — the point where extra clusters stop helping much — suggests a good k.

Cell 33 — code: Fit KMeans with k = 3 and tabulate clusters vs operators

kmeans.fit_predict assigns each operator-month to one of 3 clusters. *pd.crosstab* then shows how the seven operators distribute across the three clusters — a sanity check that the algorithm has recovered known structure.

Cell 34 — markdown: Heading — §8.1 PCA for visualisation

Cell 35 — code: Run PCA and draw two scatter plots

PCA reduces the 4-D feature space to 2-D. We then draw the same 2-D scatter twice: once coloured by operator and once by KMeans cluster. If the clusters line up with operator colours, the unsupervised model has discovered the structure on its own.

Cell 36 — markdown: Heading — §9 Machine unlearning — an industry scenario

Sets up the consultancy-loses-Airtel-data scenario described in Part E and introduces the approximate-unlearning recipe (single Newton / influence-function update) that the notebook will implement.

Cell 37 — code: Step 1–3: identify the forget set, apply the Newton update, score utility and forgetting

The forget mask selects every Airtel row. We project all rows through the original Lasso's fitted preprocessor, compute the (regularised) Hessian H of the squared-error loss on the full training data, sum the per-row gradients on the forget set, and take one Newton step *delta_theta = inv(H) · grad_sum* to update the weights. The updated coefficients are wrapped in a thin Pipeline-compatible object so that *.predict()* still works. Two metrics get reported side by side for the original and unlearned models: R^2 on retained operators (should stay essentially unchanged) and MAE on Airtel rows (any change comes only from weight-encoded information about Airtel).

Cell 38 — markdown: Heading — §9.1 Coefficient diff

Cell 39 — code: Build a coefficient comparison DataFrame

coef_table is called for the original and the unlearned Lasso. Subtracting and ranking by absolute change shows exactly which weights were moved by the Newton step — typically the *operator_Airtel* one-hot and the other operator one-hots that had to be re-balanced.

Cell 40 — markdown: Heading — §9.2 Visual check — predictions for Airtel before and after

Cell 41 — code: Line chart: actual vs original prediction vs unlearned prediction (Airtel only)

If unlearning worked, the red dashed line (unlearned) departs from the black solid line (actual) in the direction of whatever Airtel-specific information was held in the weights, while the blue dashed line (original) tracks the actual curve more tightly. The size of the gap reflects how much of the prediction came from operator-specific weights vs from the strong lag input.

Cell 42 — markdown: Heading — §9.3 Interpretation

Plain-English summary of what the metrics, coefficients and plot show — utility preserved, operator one-hot weights shifted, and a frank explanation of why a lag-driven model leaves a fundamental floor on how much the Airtel prediction can change. Also points to exact retraining, SISA and gradient-based unlearning as the alternative recipes.

Cell 43 — markdown: Heading — §10 Findings & limitations

One-paragraph wrap-up: market growth observations, which regression model won, that KMeans recovered domain-meaningful clusters, and the approximate-unlearning result. Limitations call out the small sample size, the assumption-only columns we excluded, and the local-quadratic assumption behind the Newton step.

Part G — How to read the printed outputs

When you run the notebook end-to-end, look for these checkpoints:

- **After cell 7:** a tidy DataFrame whose first column is the operator and second is a January-2008 date — a sign that the reshape worked.
- **After cell 9:** a missing-values count. Most columns should be 0; *net_addition* and *growth_rate* have one missing entry per operator (the first month).
- **After cell 14:** Grameenphone's line should dominate the chart, climbing from ~17 to ~56 million.
- **After cell 26:** a metrics DataFrame. All three models should score very high R^2 (close to 1) because the lag feature does most of the work; CV R^2 should also stay close to test R^2.
- **After cell 32:** an elbow plot. Inertia falls quickly until $k \approx 3$ and then flattens.
- **After cell 35:** two scatter plots. The colour patterns should look similar — that is the visual confirmation that KMeans rediscovered the operator structure.
- **After cell 37:** a 2-column metrics table. The retained-operator R^2 should be essentially identical between the original and the approximately-unlearned model (utility preserved). The Airtel MAE will move only slightly because the original Lasso already shrank Airtel-specific weights toward zero and the bulk of Airtel's prediction comes from the *lag_1* input — a structural property of the dataset, not a sign that unlearning failed.
- **After cell 39:** a coefficient diff table. The largest absolute changes should sit on the operator one-hots, with *operator_Airtel* typically at the top — visible evidence that the Newton step moved exactly the parameters that held Airtel-specific information.
- **After cell 41:** the Airtel line plot. The unlearned (red) line should depart from the actual (black) line in the direction of the weight-encoded Airtel signal that was removed; the original (blue) line stays closer because it still has that signal.

Closing

If anything in the notebook still feels mysterious after reading the matching section here, the standard next steps are: (1) print the intermediate DataFrame, (2) check its shape and dtypes, and (3) inspect a handful of rows. The whole scikit-learn API was deliberately designed so that any object can be probed with *.fit*, *.predict*, *.named_steps* and *.coef_* — there is nothing hidden behind the pipeline.